Design and Analysis of Energy Efficient Approximate Multipliers for Image Processing and Deep Neural Network

Anupam Kumari and Roy Paily Palathinkal, *Senior Member, IEEE*

***Abstract*— Numerous obstacles in enhancing the performance of computing systems have spurred the emergence of approximate computing. Extensive studies have been reported on approximate computing to develop high-performance, energy-efficient hardware designs tailored to error-resilient applications. In this brief, we proposed 8-bit approximate multipliers with 15 levels of accuracy using three techniques: recursive, bit-wise, and hybrid approximation using partial bit OR (PBO). Compared to the existing multipliers, investigated designs have significantly improved the area, power, delay, Power Delay Product (PDP), and Power Area Delay Product (PADP) by 41.68%, 73.16%, 35.57%, 72.65%, and 75.42% respectively on average. On resemblance with the accurate multiplier, the area, power, delay, PDP, and PADP were enhanced by 54.41%, 57.57%, 25.73%, 60.14%, and 74.33% correspondingly on average. Peak Signal-to-Noise Ratio (PSNR) and Structural Similarity Index Measure (SSIM) values surpassing (30 dB, 94%), (31 dB, 96%), and (26 dB, 95%) by applying them to benchmarks in image smoothing, edge detection, and image sharpening successively. Moreover, upon scrutinizing the efficacy of multipliers in hardware implementations of deep neural networks attaining the performance exceeding 95%. The obtained results confirm that suggested multipliers are well-suited for their widespread applications.**

***Index Terms*— Approximate computing, approximate multiplier, energy efficient, image processing, neural network.**

I. Introduction

With the rapid advancement in image processing and artificial intelligence necessitates the integration of complex hardware in existing systems. Devices are normally characterized by limited computing capability and they are area, power, and speed-constrained [1]. To address these challenges, the emergence of inexact processing has become a prominent strategy. Due to their ability to withstand slight accuracy reductions with negligible impact on performance, approximate computing has proven highly effective in error-resilient applications such as neural networks, scientific computing, data analytics, image processing, speech processing, and multimedia applications [2], [3]. These applications require intense matrix multiplications. Therefore, it is imperative to develop hardware-efficient approximate multipliers. Harnessing error-robust capability presents a unique opportunity to achieve heightened efficiency in power, speed, and area by making calculated compromises on accuracy [4].

In recent years, improving the hardware efficiency of arithmetic circuits using approximate computing has become a key research area in digital circuits. Hence, enhancing their efficiency is crucial. Multiplication is more complex than addition. Extensive approaches have been investigated to obtain highly efficient approximate multipliers [5], [6], [7], [8], [9], [10], [11]. Approximate computing methods are divided into circuit, architectural, and software levels [3]. Circuit level includes voltage and frequency scaling [12]. Architectural level contains RISC-V ISA extension [13], [14], approximate accelerator and storage [15]. Software level mentioned Loop and Code Perforation [16]. However, the most favourable approach is imprecise numerical and logical hardware design [3], [17], [18], [19]. The latest work by Rashidi [20], where two types of multipliers were designed using a 4:2 approximate compressor and two 4-bit adders with minimizing computational cost was proposed. Multipliers are fundamental building blocks of digital systems. However, exact multipliers often impose considerable demands in terms of area, critical path, and power consumption. While extensively employed in neural networks, image processing, and various applications, their efficiency remains constrained. To address these limitations, approximate computing has gained widespread adoption and significantly improved diverse applications. In the realm of carry-overlook architecture, ignoring the carry introduces a notable increment in error. Hence, there is a pressing need for substantial modifications to existing carry-disregard methodologies.

This paper introduces a new insight: when ignoring the carry, the use of the OR gate for sum computation instead of the conventional XOR gate leads to a significant reduction in errors. Apart from this, the presented designs give exact results for any number of carry disregard bits if any operand of multipliers value is 2^n ($n \epsilon W$). An intriguing aspect lies in the fact that elements of filters like the Sobel, Prewitt, Laplacian, and numerous others can all be expressed as 2^n. Thus, employing proposed multipliers for edge detection and image sharpening invariably yields accurate results. The primary contributions of this studies can be encapsulated as:

1) By adopting three new techniques for designing 15 approximate multipliers: i) recursive approach using PBO for less error-resilient, ii) bit-wise approximation using PBO for high error-resilient, and iii) hybrid approximation for balanced error.

Received 30 April 2024; revised 1 August 2024 and 17 September 2024; accepted 5 October 2024. Date of publication 22 October 2024; date of current version 30 January 2025. This article was recommended by Associate Editor J. Kulkarni. *(Corresponding author: Anupam Kumari.)*

The authors are with the Department of Electronics and Electrical Engineering, Indian Institute of Technology Guwahati (IIT Guwahati), Guwahati 781039, India (e-mail: a.kumari@iitg.ac.in; roypaily@iitg.ac.in).

Digital Object Identifier 10.1109/TCSI.2024.3479279

TABLE I
SUMMARY OF STATE OF THE ART APPROXIMATION TECHNIQUES FOR MULTIPLIERS

Reference	Approaches	Tech (nm)	No. of Bits	Descriptions	Reported (8x8) Area ($(\mu m)^2$) (min, max)	Power (μW) (min, max)	Delay (ns) (min, max)	Error (MRED) (min, max)
TCAS-I 2023 [3]	Recursive	45	8, 16	Designed 8x4 using carry disregard 13 Design of 8*8 using two 8*4 multipliers 15 design of 16x16 using four 8x4 multipliers	**(156, 301)**	(41, 86)	(0.48, 0.76)	(0, 0.214)
DSD 2022 [17]	Recursive	45	8	Designed 8x4 using carry disregard 2 design of 8x8 using two 8x4 multiplier	**(279, 296)**	(81,99)	(0.64, 1.04)	(0, 0.0018)
TETC 2020 [18]	Recursive	45	2, 8	Design of NOR based approximate 2 HA, 1FA 2 design of 4x4 using NXHA1, NXHA1, NXFA 3 design of 8x8 using 4x4	(302, 419)	(143, 182)	(0.77, 0.84)	(0.001, 0.028)
ICCAD 2016 [21]	Recursive	45	2, 4, 8, 16	5 design of 2x2 multipliers 7 design of 1-bit adder	(222, 324)	**(12.5, 16.5)**	(1.6, 1.83)	(-)
ISQED 2014 [19]	Recursive	45	2, 8, 16	4x4 and 8x8 using carry-in prediction Accuracy configurable for 4x4 multipliers 16x16 using three 8x8 and four 4x4 multipliers	313.63	38.66	1.41	0.2011
VLSID 2011 [22]	Recursive	45	2, 8, 16	Design of accurate and inaccurate 2x2 multiplier Error detector and corrector adder for 2x2 Design of 4x4 using 2x2, Design of 8x8 using 4x4	366	37.704	1.40	0.02925
TETC 2022 [23]	Recursive	14	4, 8, 16, 32	Design of half OR based 4x4 multiplier Designed 8x8 using 4x4	**(70, 81)**	(94, 109)	**(0.174, 0.22)**	(0, 0.029)
TENCON 2018 [24]	Recursive	40	8	HA, FA sum using OR gate and considered correct carry 4x4 using OR gates on lower side and exact on upper side	(328, 700)	(95, 193)	(0.89, 1.28)	(0, 0.018)
TETC 2021 [5]	Recursive	28	2, 8	2 design of 2x2 multiplier 3 design of 8x8 using two 2x2 multiplier	**(143, 185)**	(126, 160)	(0.75, 1.09)	(0, 0.0134)
Integration, the VLSI Journal 2022 [25]	Recursive+ Compressor	45	4, 8	2 type of approximate 4x4 multipliers 8x8 design recursively using 4x4 multiplier	(249, 359)	(335, 453)	**(0.227, 0.249)**	(0.0019, 125)
TCAS-1 2023 [6]	Compressor	28	8, 16	3 types of proposed approximate compressor 2 proposed multiplier using compressor	(318 , 331)	(73, 76)	**(0.09, 0.1)**	(0.151, 0.509)
TVLSI 2021 [7]	Compressor	28	8, 16	Analysis of low and high accuracy 4:2 compressor design of 5 optimize critical path and 2 hybrid	(498, 582)	(503, 609)	**(0.164, 0.165)**	(0.0018, 0.00047)
IESL 2018 [26]	Compressor	65	8	Proposed 4-2 Compressor and Error Recovery Module 8x8 design using exact, approx 4:2, HA, and FA	(510, 674)	(164, 215)	(2.7, 3.04)	(-)
IESL 2023 [27]	Compressor	90	8, 16	Multiplier with truncated, approximated, and exact region Error corrector logic also used to reduce error	(926, 1261)	(27.6, 46)	(2.7, 2.93)	(0, 0.0014)
Integration, the VLSI Journal 2024 [20]	Compressor	180	8	Approximate compressor, 4-bit adder (FA1, FA2) design Two multiplier designed using compressor, FA1, and FA2	(2115, 4920)	(-)	(1.451, 5.46)	(0.0251, 0.16)
IET 2019 [28]	Compressor	90	8	7 multiplier using reference and proposed 4-2 compressor	(-)	**(9.42, 15.09)**	(0.57, 0.89)	(0.009, 0.13)
ISES 2021 [29]	Compressor	90	8	Proposed two multipliers using the 4-bit bypass, 4-bit OR, 7-bit exact operation	(832, 916)	(26.97, 30.4)	(2.82, 2.83)	(0.59, 0.94)
TCAS-I 2018 [30]	Compressor	40	8, 12, 16, 20	High order compressor using lower order compressors 4 multipliers have designed 2 using PPM truncation	(171, 524)	(532, 1436)	(0.375, 0.5)	(-)
IEEEAcces 2020 [31]	Compressor	45	8, 16	modified Dual-stage 4 : 2 compressor multiplier	(2468, 3979)	(51, 101)	(0.33, 0.63)	(0, 0.0487)
AEU 2019 [32]	Compressor	45	8	Nine 8*8 have designed using 4-2 compressor with and without truncation	(614, 3979)	(28.8, 101)	(0.29, 0.625)	(0, 0.0292)
TSC 2021 [33]	Logrithmic	45	8, 16, 32	Non-iterative LM architecture Design of DR-ALM	(230, 333)	(27, 74)	(0.52, 0.8)	(0.0027, 0.202)
TCAS-1 2021 [8]	Logrithmic	45	8, 16	Logarithm Conversion of Operands Summation and Antilogarithm Conversion	(313, 359)	**(11.5, 13.8)**	(0.85, 0.91)	(0.12, 0.13)
TCAS-1 2018 [9]	Logrithmic	45	8, 16	ALM Using LOA, ALM Using MAA3, ALM Using SOA IALM-S, IALM-SL, IALM-SM	**(108, 350)**	**(16, 115)**	(0.39, 0.8)	(0.03, 0.165)
TOC 2021 [34]	Logrithmic	28	8, 16	Proposed nearest one detector circuit ILM and priority encoder	**(255, 287)**	(50, 54)	(1.64, 1.68)	(0.03, 0.0869)
TCAS-II 2023 [1]	Booth	65	8	Extended Booth design with default mode and reduced mode PO2 multiplier	(186, 926)	(38, 253)	(0.42, 1.14)	(-)
TOC 2017 [35]	Booth	45	8, 16, 32	Radix-4 Booth encoding method 1 with XOR func Radix-4 Booth encoding method 2 with one XOR-2	(405, 671)	(88, 167)	(0.55, 0.62)	(0.0005,12.91)
TVLSI 2019 [10]	Truncation	45	8, 16, 32	Leading-One Detector Unit for 8-bit operands accuracy configurable TOSAM with three modes	(143, 417)	(104, 360)	(0.57, 1.22)	(0, 14.4)
TCAS-II 2023 [36]	Truncation	7	8	Multiplier with constant truncated region, Error compensation and exact part	**(6.23)**	**(7.98)**	**(0.187)**	(0.035)
TCAS-I 2020 [11]	Truncation	45	8, 16	Obtain static information after truncation Utilizing the proposed padding scheme	(451, 757)	(96, 163)	(0.59, 0.75)	(-)

2) Our strategy excels in power, area utilization, delay, PDP, PADP, and accuracy compared to the latest existing multipliers.
3) Evaluating the performance of the designs across image smoothing, edge detection, and image sharpening tasks.
4) FPGA implementation of handwritten digit detector using approximate multipliers with MNIST dataset.

The remaining parts of the paper are organized as follows: section II reviews related works. Section III provides background on designed 8-bit multipliers. Section IV details the design and characteristics of investigated multipliers. Experimental hardware setup and results analysis are acquainted in Section V. Section VI comprehensively compares the trade-off between area, power, delay, PDP, and PADP versus error for intended designs and literature multipliers. Section VII explores the integration of approximate multipliers in image processing and deep neural networks with performance comparisons against recent studies. Finally, section VIII comprises the conclusions drawn from this study.

II. RELATED WORK

Multipliers serve as essential components within digital systems. It comprises three steps: Partial Product (PP) computation using bit-wise AND operations, PP reduction, and PP addition. Researchers have employed various PP addition, reduction, and accumulation methods. We endeavoured to summarise these in Table I. The array multiplier has a uniform, straight, and modular architecture. This architecture is more comprehensible than Wallace and Dadda's architecture. Recursive architecture mostly has less power and area.

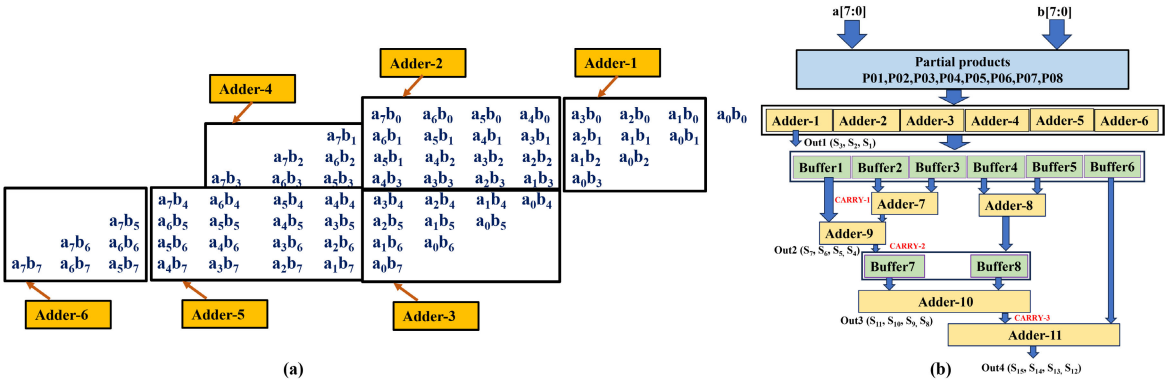

Fig. 1. Structure of exact 8 × 8 unsigned multipliers: (a) generation of 64 partial products by multiplying each bit of the multiplicand with each bit of the multiplier and (b) The parallelized partial product addition minimizes critical path delay by strategically placing buffers along the critical path to balance delays and enhance overall performance.

Correspondingly, compressor-based design has the highest area and the least delay. Truncation multipliers take advantage of all bits that are not equally important. On the other hand, we can apply approximation during the PP accumulation stage using an approximate compressor. Array multiplier using approximate Half-Adder (HA), Full-Adder (FA) and approximate compressor is a conventional way for reducing PPs. Apart from this, Author's also incorporated smaller multipliers for the formulation of larger scale multiplier recursively [3], [5], [17], [18], [19], [21], [22], [23], [24]. In logarithmic multipliers, the multiplication result was calculated by adding the approximation of the logarithm of operands and by taking the antilogarithm of the sum [8], [9], [33], [34]. We observe that the Logarithmic multiplier has the least area and power. Booth architecture substitutes the input bit into an alternate representation. Several approaches for booth multipliers utilize mixed, hybrid radix, or others were provided in [1] and [35]. Work related to reducing the complexity of PPs addition using approximate compressor was explored in [6], [7], [26], [27], [28], [29], [30], [31], and [32]. About truncation multipliers, the lower m-bit of the partial products were ignored vertically or horizontally and for some suitable error corrector circuits were used to mitigate the truncation error [10], [11], [36]. Additionally, voltage scaling has been explained in [12] and Mitchell's algorithm to Convert the operands to the logarithmic domain has been explored in [37]. We have also tried to summarized the minimum and maximum values of area, power, delay, and error in Table I.

III. Background

A standard 8-bit unsigned multiplier, where the critical path is significantly affected by the partial product adder (PPA). In the 8 × 8 multiplier, a total of 64 partial products are generated. To sum up these partial products using the standard approach, we need seven ripple carry adders (RCAs). Four RCAs operate in parallel in the first stage, each tasked with adding 9 bits, and its critical path is $8 \times T_{carry} + Max(T_{Sum}, T_{carry})$. Two RCAs run concurrently after the completion of the first stage, each handling 11-bit additions, and their critical path is $10 \times T_{carry} + Max(T_{Sum}, T_{carry})$. Finally, a single RCA processes a sum of 15 bits after finishing the second stage, and the critical path is $14 \times T_{carry} + Max(T_{Sum}, T_{carry})$. Therefore, the critical path for the partial product addition of 8 × 8 multiplier is given by the addition of these three critical paths. Generally, n×n multiplier has n^2 partial product factor. The total stage required to add partial product $\log_2(n)$. As a result, adding the critical path of $\log_2(n)$ stages will be very high for higher n. To minimize critical path delay, Adder-1 to Adder-6 is employed to sum 64 partial products, as shown in Figure 1(a). To sum the outputs of Adder-1 through Adder-6, buffers are interspersed between them to further decrease the critical path, as depicted in Figure 1(b). The LSB bit is passed directly to the output. Adder-2 to Adder-6 operate parallel with their sums and carry buffered to the next stage via Buffer-2 to Buffer-6 correspondingly. Adder-1 is also executed simultaneously with Adder-2 to Adder-6 and directly feeds the output to S_1, S_2, and S_3, and its carry is stored in Buffer-1. Buffer2 and Buffer3 are added with Adder-7, whose output is combined with the carry of Adder-1 using Adder-9 to obtain S_4, S_5, S_6, S_7. Buffer4 and Buffer5 are added with Adder-8. The resulting sum and Adder-9's carry are buffered into the third stage using Buffer8 and Buffer7 respectively. Buffer8 and Buffer7 values are added with Adder-10 yielding S_8, S_9, S_{10}, S_{11}. The carry of Adder-10 is then added to Buffer6 via Adder-11 resulting in S_{12}, S_{13}, S_{14}, S_{15}. Incorporating 8 buffers for an 8 × 8 multiplier lays the groundwork. Yet for a higher bit multiplier, the escalating need for buffers increases the area substantially. In high-error-resilient applications, opting for an approximation of this circuit will work best as it minimizes the number of buffers. In contrast, for applications demanding precision or exhibiting lower error resilience, the implementation of smaller-scale multipliers works fine. Utilizing the distributive property as demonstrated in equations (1) and (2). Large-scale multipliers can be subdivided into smaller ones. Specifically, four 4 × 4 or two 8 × 4 multipliers can construct an 8 × 8 multiplier. However, this study exclusively focused on implementing 8 × 4 multipliers, omitting discussions on 4 × 4 configurations.

$$A \times B = A_H \times B_H + A_H \times B_L + A_L \times B_H + A_L \times BL \tag{1}$$

$$A \times B = A \times B_H + A \times B_L \tag{2}$$

Figure 2 reveals our design approach, which consists of two 8 × 4 multipliers to construct an 8 × 8 multiplier. Each 8 × 4 multiplier operates in parallel and independently, calculating the summation of their outputs using an adder. To further reduce the critical path delay in the 8 × 8 multiplier design, a buffer has been introduced for summing the outputs of the 8 × 4 multipliers. Designing one 8 × 4 multiplier

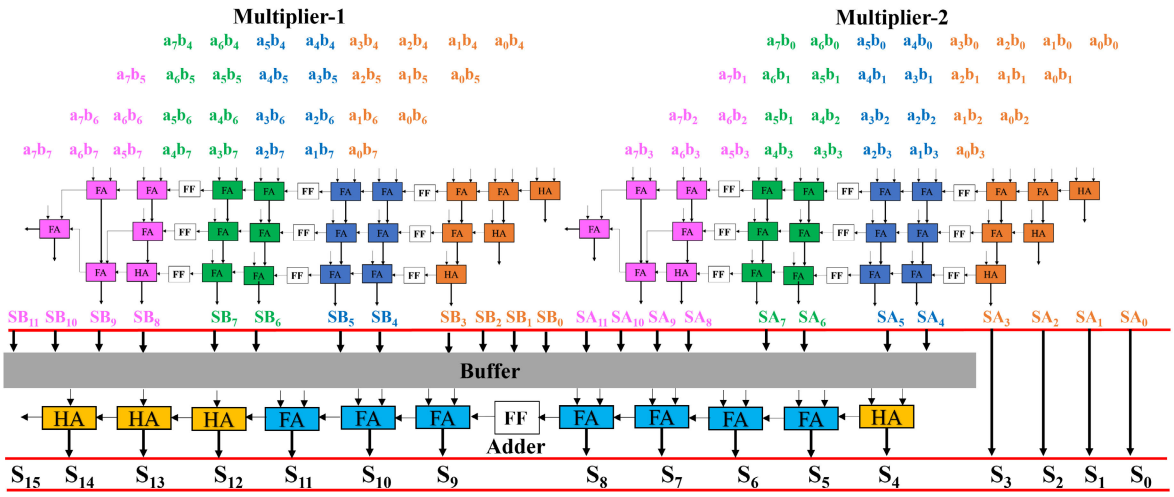

Fig. 2. Enhanced architecture: a glimpse into the design of an unsigned 8 × 8 multiplier, ingeniously designed from two unsigned 8 × 4 multipliers.

needs 20 full adders (FA) and 4 half adders (HA), while the addition of both 8 × 4 multipliers outputs demands 7 FAs and 4 HAs. Consequently, the total hardware requisite for an 8 × 8 multiplication comprises 47 FAs and 12 HAs. Each 8 × 4 multiplier mandates nine 1-bit buffers. To store the outputs of both multipliers, a total of eighteen 1-bit buffers are required. Additionally, one buffer is inserted when merging the outputs of the two 8×4 multipliers. Therefore, the total number of buffers required is $9 \times 2 + 18 + 1 = 37$. The foundation of the provided multiplier prioritizes a less critical path and minimal power consumption. This is achieved by employing smaller-scale multipliers configuration and inserting buffers to mitigate critical path delays further. Due to the inherent carry dependency, the introduction of additional buffers becomes necessary for higher bit multipliers. Thus, the paper's primary goal is to employ approximate techniques to break the dependency on carry, such that the design will be balanced between hardware efficiency and accuracy.

IV. Proposed Approximate Multipliers

This paper introduces 15 approximate multipliers using three approaches: i) a recursive multiplier utilizing 8 × 4 multipliers, aimed at precise or less error-resilient applications, guided by Figure 2; ii) bit-wise approximation designed for highly error-resilient scenarios, mentored by Figure 1; iii) a hybrid approximation utilizing two 4:2 approximate compressors to strike a balance between hardware efficiency and accuracy. The propagation of carry significantly contributes to higher delays, area requirements, and power consumption. Therefore, disregarding carries in the given multipliers can substantially enhance hardware efficiency, albeit at the cost of accuracy. However, to compensate for the absence of carry, modifications are necessary in the sum function to enhance accuracy. In Table II, two inputs of each 2-bit have been considered, and their XOR, OR addition results have been computed. We've calculated the Error Probability (P_E), and Mean Error (E_{mean}) using equation 3 [30]. We get $XOR(P_E) = 31/256$, $XOR(E_{mean}) = 96/256$ and $OR(P_E) = 31/256$, $OR(E_{mean}) = 48/256$. Through an analysis of XOR and OR gate's mean error, it is observed that the OR gate yields superior results for the sum function

TABLE II
Comparing Error Distance: XOR Gate Versus OR Gate for Sum Function With Carry Disregard

Input-1	Input-2	XOR result	OR result	Exact result	Err_i XOR	P (Err_i) XOR	Err_i OR	P (Err_i) OR
00	00	00	00	00	0	-	0	-
00	01	01	01	01	0	-	0	-
00	10	10	10	10	0	-	0	-
00	11	11	11	11	0	-	0	-
01	00	01	01	01	0	-	0	-
01	01	00	01	10	2	9/256	1	9/256
01	10	11	11	11	0	-	0	-
01	11	10	11	100	2	3/256	1	3/256
10	00	10	10	10	0	-	0	-
10	01	11	11	11	0	-	0	-
10	10	00	10	100	4	9/256	2	9/256
10	11	01	11	101	4	3/256	2	3/256
11	00	11	11	11	0	-	0	-
11	01	10	11	100	2	3/256	1	3/256
11	10	01	11	101	4	3/256	2	3/256
11	11	00	11	110	6	1/256	3	1/256

when carries are disregarded.

$$P_E = \sum_{i=1}^{n} P(Err_i)$$
$$E_{mean} = \sum_{i=1}^{n} P(Err_i)(Err_i) \quad (3)$$

When comparing the mean error values, it becomes evident that $OR(E_{mean})$ is consistently smaller than $XOR(E_{mean})$ for the possible input values. Therefore, it is preferable to use the OR gate instead of the XOR gate. However, for higher-order bits, utilizing the OR gate for addition and overlooking the carry will result in an increased error. To address this, we devised two 4:2 approximate compressors. This section investigates how each approximate multiplier exhibits distinct accuracy levels, area requirements, power consumption, and delays.

A. Proposed Approximation for 8× 4 Multiplier

Figure 4 illustrates four types of 8 × 4 multipliers using PBO adders, named as PBO-3, PBO-5, PBO-7, and PBO-10. In all four variants, approximation begins from the second

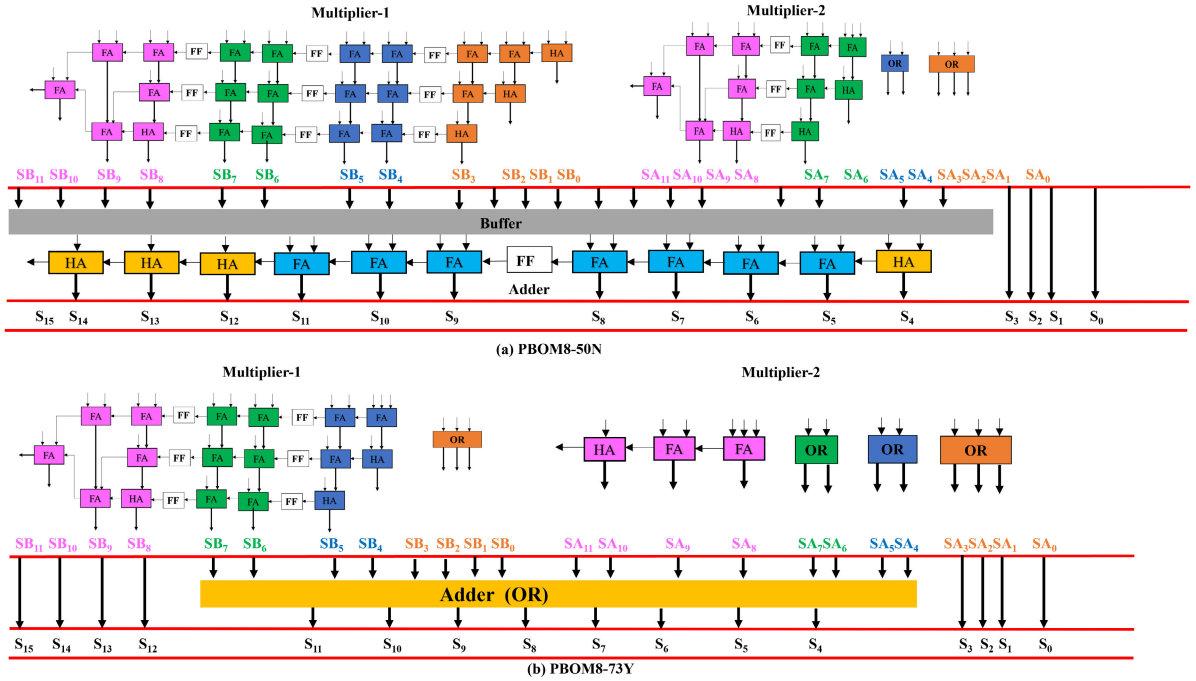

Fig. 3. Schematic depiction of the approximate 8-bit unsigned multiplier design utilizing two exact/approximate 8 × 4 multipliers: (a) retaining the final adder as exact and (b) utilizing an approximate final adder.

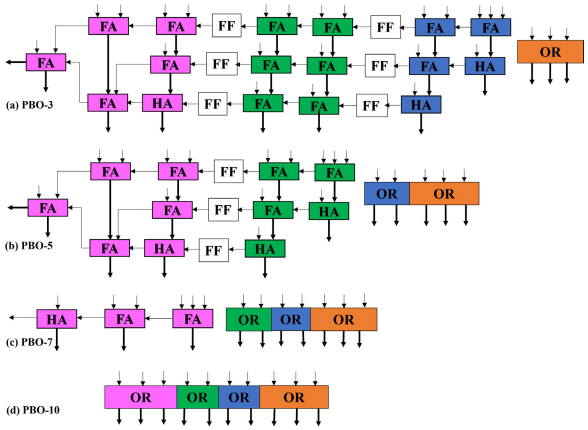

Fig. 4. Interpretation of the approximate 8 × 4 multiplier architecture featuring four variants of approximation: (a) PBO-3, (b) PBO-5, (c) PBO-7, and (d) PBO-10.

column onwards, as the output of the first column does not generate any carry. For instance, in PBO-3 (depicted in Figure 4(a)), carry is disregarded from columns 2 to 4, resulting in the parallel operation of columns 1 to 5. The sum for columns 2 to 4 is calculated using OR gates instead of XOR gates. An exact 8 × 4 multiplier requires 20 FAs, and 3 HAs and provides output after 3 clock cycles. On the other hand, PBO-3 demands 14 FAs, 3 HAs, and one 3-input OR gate, yielding output after 2 clock cycles. Accordingly, PBO-3 reduces area and latency by 6 FAs and 1 clock cycle with the addition of a 3-bit OR gate. In PBO-5, illustrated in Figure 4(b), carry is disregarded from columns 2 to 6, with the sum calculated using OR gates. Designing PBO-5 requires 8 FAs, 3 HAs, and one 5-bit OR gate, providing output after one clock cycle. Thereupon, the area and latency are reduced by 12 FAs and 2 clock cycles, albeit with a slight increase in error. PBO-7 represents an independent operation of columns 1 to 8, as shown in Figure 4(c). No carry is generated in columns 1 to 8, and the sum is computed using OR gates for columns 1 to 7. Designing PBO-7 necessitates 2 FAs and 1 HA, with output provided without any clock delay. Thus, the area is reduced by 18 FAs and 2 HAs, and latency is reduced by 3 clock cycles. Critical path delay is also decreased as the critical path of the exact 8 × 4 multiplier, stemming from columns 9 to 11 (5 FAs, 1 HA), is now reduced to (2 FAs, 1 HA). PBO-10, depicted in Figure 4(d), columns 2 to 11 are approximated using OR gates. No carry is generated for columns 2 to 11, and the sum is computed using a 10-bit OR gate. The area (20 FAs, 3 HAs) of the exact 8× 4 multiplier is thus replaced by a single 10-input OR gate. In conclusion, PBO-3 and PBO-5 exhibit minimal approximation but greater hardware complexity, while PBO-7 and PBO-10 feature higher approximation but reduced hardware complexity.

B. Proposed Approximation for 8× 8 Multiplier Using 8× 4 Multiplier

Figure 3 showcases the proposed approximation schemes for 8×8 multipliers using two 8×4 multipliers and one adder. Seven approximate multipliers are introduced utilizing two different combinations of 8×4 multipliers with the exact adder, in which one design is depicted in Figure 3(a). Additionally, one approximate multiplier is designed using two approximate 8×4 multipliers and one approximate adder, as depicted in Figure 3(b). Exclusion of configurations such as PBOM8_10, PBOM8_20, PBOM8_40, PBOM8_41, PBOM8_42, etc. are done because these designs are not able to remove buffers between combinational paths. Higher configurations like PBOM8_77, PBOM8_107, and PBOM8_1010 are excluded because it is having complex designs for higher errors. In Table III, suitable combination of two distinct 8×4 multipliers and one adder is elucidated to further reduce area, power, delay, and error. While the architecture in Figure 2 has

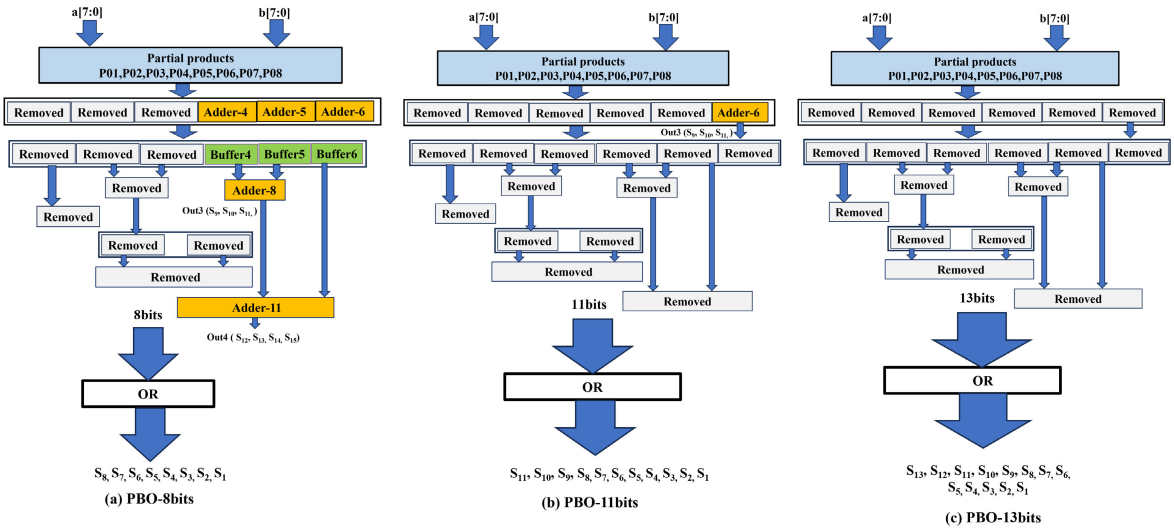

Fig. 5. The architecture of a bitwise approximation for 8-bit unsigned 8× 8 multipliers for high error resilient application.

TABLE III
PROPOSED 8 × 8 MULTIPLIER UTILIZING TWO EXACT/APPROXIMATE 8 × 4 MULTIPLIERS AND ADDER

Design PBO	Multiplier-2 Approximation	Multiplier-1 Approximation	Adder Approximation
PBOM8-30N	Column 2 to 4	Exact	Exact
PBOM8-50N	Column 2 to 6	Exact	Exact
PBOM8-33N	Column 2 to 4	Column 2 to 4	Exact
PBOM8-53N	Column 2 to 6	Column 2 to 4	Exact
PBOM8-73N	Column 2 to 8	Column 2 to 4	Exact
PBOM8-75N	Column 2 to 8	Column 2 to 6	Exact
PBOM8-105N	Column 2 to 11	Column 2 to 6	Exact
PBOM8-77N	Column 2 to 8	Column 2 to 8	Exact
PBOM8-107N	Column 2 to 11	Column 2 to 8	Exact
PBOM8-1010N	Column 2 to 11	Column 2 to 11	Exact
PBO-73Y	Column 2 to 8	Column 2 to 4	Column 5 to 11

already been fine-tuned to reduce delay through the buffer. Our main motivation has been on enhancing power and area efficiency primarily through approximate methods. In the given architecture, the hardware complexity of the multiplier is divided into three parts: (i) hardware because of Multiplier-1 (ii) hardware because of Multiplier-2, and (iii) hardware because of the final Adder and Buffer. To achieve the best design, reducing the complexity of these three parts is crucial. Table III outlines the proposed approximate 8×8 multiplier using an exact/approximate 8×4 multiplier and adder named PBOM8-xyz by considering different levels of accuracy and hardware complexity. Here, x and y denote the number of columns of Multiplier-2 and Multiplier-1 approximated, while z indicates whether the final Adder is exact (N) or approximated (Y). For example, PBOM8-53N implies that the carry of columns 2 to 6 is ignored in Multiplier-2, the carry of columns 2 to 4 is ignored in Multiplier-1 and the Adder is exact. Similarly, PBOM8-73Y, illustrated in Figure 3(b), signifies that columns 2 to 8 in Multiplier-2 and columns 2 to 4 in Multiplier-1 are approximated using OR gates without carry generation. The final Adder is replaced by an OR gate. In Table III, 11 designs have been described. However, eight multipliers have been opted, namely PBOM8_30N, PBOM8_30, PBOM8_33N, PBOM8_53N, PBOM8_73N, PBOM8_75N, PBOM8_105N, and PBOM8_73Y, while discarding PBOM8_77N, PBOM8_107N, and PBOM8_1010N. These three designs are prone to higher error due to the approximation of higher significant digits, particularly in Multiplier-1. Employing complex architectures for designing high-approximation circuits is not advisable. Therefore, the multiplier depicted in Figure 1 has been approximated for higher error-resilient applications.

C. Proposed Approximation for 8 × 8 Multiplier for High Error Resilient Application

The current subsection outlines an architecture for bitwise approximate multipliers inspired by the multiplier interplayed in Figure 1. This architecture involves the development of six distinct approximate multipliers, namely PBOM8_8bits, PBOM8_9bits, PBOM8_10bits, PBOM8_11bits, PBOM8_12bits, and PBOM8_13bits. Each of these multipliers represents a different level of approximation applied during the multiplication process. For instance, PBOM8_8bits illustrated in Figure 5(a), PBOM8_9bits, and PBOM8_10bits have varying levels of approximation. In PBOM8_8bits, the carry from column 2 to column 9 is omitted, while PBOM8_9bits extends this omission to column 10, and PBOM8_10bits extends it further to column 11. In these cases, an OR gate is employed to calculate the sum for the specified columns instead of using XOR gates. The addition of the remaining bits is likely executed using an exact adder to maintain precision. PBOM8_11bits, shown in Figure 5(b), the carry from column 2 to column 12 is ignored, and the sum is calculated using an 11-bit OR gate. Other most significant 4-bit additions have been done using exact addition. For this design Adder-6 and an 11-bit OR gate are required. In the same way, PBOM8_12bits can be understood. For PBOM8_13bits, as depicted in Figure 5(c), the carry from column 2 to column 14 has been disregarded, and the sum was computed using a single 13-bit OR gate. Therefore, all hardware components can be removed for this design and replaced with a single 13-bit OR gate. Neglecting the carry for higher significant bits inevitably results in increased error.

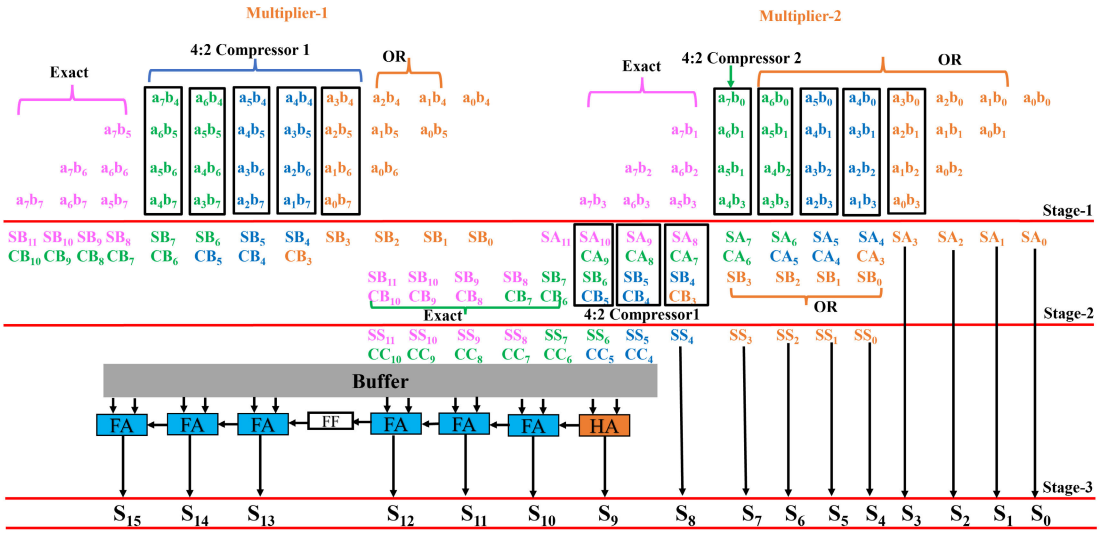

Fig. 6. The schematic portrays a hybrid approximation for an 8-bit multiplier, employing two approximate 8×4 multipliers. This design integrates a combination of compressors, OR gates, and exact operations, culminating in a three-stage completion.

Thus, the concept of an approximate compressor arose to mitigate this error, as elaborated in the subsequent subsection.

D. Hybrid Multiplier

Two types of 4:2 approximate compressors have been proposed, each exhibiting unique error characteristics. One had a minimal error, while the other had a slightly higher one. In the comparison presented in Table IV, the Error of XOR, OR, Compressor-1 (Comp-1), and Compressor-2 (Comp-2) have been examined. The logical equations governing the sum and carry functions for both compressors have been provided below.

$$Sum1 = a \oplus b \oplus c \oplus d \tag{4}$$

$$Carry1 = ab + ac + ad + bc + bd + cd \tag{5}$$

$$Sum2 = a + b + c + d \tag{6}$$

$$Carry2 = abc + acd + bcd \tag{7}$$

In order to compute the P_E, and E_{mean} equation mentioned in 3 has utilized. We get $XOR(P_E) = 67/256$, $XOR(E_{mean}) = 136/256$, $OR(P_E) = 67/256$, $OR(E_{mean}) = 81/256$, $Comp-1(P_E) = 1/256$, $Comp-1(E_{mean}) = 2/256$, $Comp-2(P_E) = 55/256$, $Comp-2(E_{mean}) = 55/256$. Hence, we can say that Compressor-1 consists of the least and XOR has the highest mean error. The Critical path delay for Compressor-1 primarily stems from the carry logic. On the other hand, Compressor-2's critical path delay arises from the sum logic. In terms of area utilization, Compressor-1 surpasses Compressor-2 due to its higher gate count and complexity. In this context, "hybrid" signifies the amalgamation of multiple approximation methods in the design of an 8 × 8 multiplier. The hybrid multiplier is structured using two approximate 8 × 4 multipliers operating in three stages. While Multiplier-1 and Multiplier-2 operate concurrently, the stage 2 compressor is activated once both multipliers complete their tasks. As a result, the critical path delay in the hybrid multiplier primarily arises from these two stages. To mitigate hardware complexity while maintaining a satisfactory level of accuracy in the hybrid multiplier, the functionalities of compressor 1 and compressor 2 have been intertwined. Multiplier-1 is designed using column 2 to column 3 are designed by OR gate, columns 4 to 8 are calculated by compressor-1, and columns 9 to 11 is formulated by exact adder. Multiplier-2 follows a similar design approach using the OR operation for columns 2 to 7, column 8 is using compressor-2 and columns 9 to 11 using exact operations.

In the second stage, columns 1 to 4 are directly routed to the final output, while columns 5 to 8 undergo OR operations instead of compression. Columns 9 to 11 are handled by compressor-1, and columns 12 to 15 are calculated using exact additions. In the third stage, accurate addition operations are performed. Utilizing this approach, a hybrid multiplier effectively diminishes hardware complexity while simultaneously mitigating errors and achieving a favourable balance between hardware requirements and accuracy.

TABLE IV

ERROR DISTANCE COMPARISON OF 4-BIT INPUTS SUM FOR XOR, OR, COMPRESSOR-1, AND COMPRESSOR-2

Input	Err_i	$P(Err_i)$	Err_i	$P(Err_i)$	Err_i	$P(Err_i)$	Err_i	$P(Err_i)$
	XOR	XOR	OR	OR	Comp-1	Comp-1	Comp-2	Comp-2
0000	0	-	0	-	0	-	0	-
0001	0	-	0	-	0	-	0	-
0010	0	-	0	-	0	-	0	-
0011	2	9/256	1	9/256	0	-	1	9/256
0100	0	-	0	-	0	-	0	-
0101	2	9/256	1	9/256	0	-	1	9/256
0110	2	9/256	1	9/256	0	-	1	9/256
0111	2	3/256	2	3/256	0	-	0	-
1000	0	-	0	-	0	-	0	-
1001	2	9/256	1	9/256	0	-	1	9/256
1010	2	9/256	1	9/256	0	-	1	9/256
1011	2	3/256	2	3/256	0	-	0	-
1100	2	9/256	1	9/256	0	-	1	9/256
1101	2	3/256	2	3/256	0	-	0	-
1110	2	3/256	2	3/256	0	-	0	-
1111	4	1/256	3	1/256	2	1/256	1	1/256

V. EVALUATIONS AND RESULTS

A. Evaluation Metrics

The evaluation of hardware criteria primarily focuses on these main parameters: power consumption, area utilization,

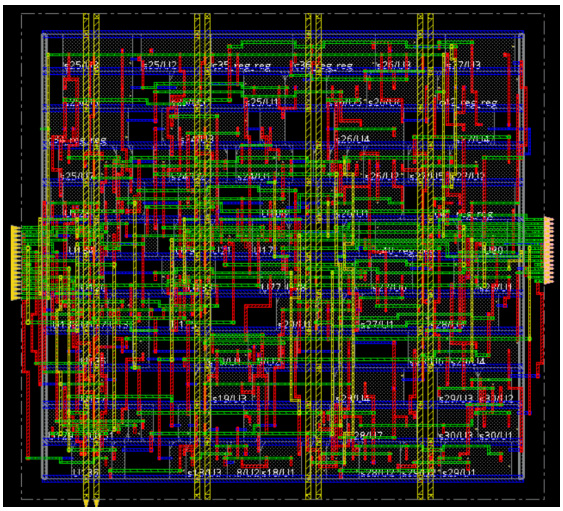

Fig. 7. Layout design of PBOM8_73Y.

and critical path delay. However, a comprehensive comparison often necessitates combining these parameters for better insights. We have compared various metrics such as PDP, PADP, Power Delay Error Product (PDEP) and Power Delay Area Error Product (PDAEP). Additionally, for error assessment the Mean Relative Error Distance (MRED), Mean Error Distance (MED), Normalised Mean Error Distance (NMED), Mean Square Error Distance (MSED), Normalized Mean Square Error Distance (NMSED), Number of Effective Bits (NoEB) and Error Rate (ER) are considered.

B. Experimental Environment

All designs are described in Verilog HDL, and their verification is conducted using Vivado Design Suite by Xilinx. Synthesis of the designs is performed on Synopsys Design Compiler (DC) utilizing UMC 40 nm technology. The layout design, from RTL to GDS, has been meticulously using the Cadence Innovus tool. The error metrics are calculated using Python programming language for the proposed approximate multipliers across all possible 2^{16} input combinations.

C. Results

For the proposed design, two critical stages of verification have been executed: post-synthesis verification and post-layout verification (Figure 7). Table V presents evaluation outcomes on comparing approximate multipliers with Exact_8bits design in terms of hardware efficiency and error criteria. Proposed multipliers show improvements in the average area (54.41%), power (57.57%), delay (25.73%), PDP (60.14%), and PADP (74.33%). Compared to all investigated designs, PBOM8_30N has the highest area, power consumption, critical path delay, PDP, and PADP which is enhanced by 9.22%, 8.396%, 0%, 8.38%, and 16.84% respectively. In contrast, PBOM8_30N has the least MRED, MED, NMED, MSED, NMSED, NoEB, and ER which are 0.00102, 3.3437, 0.000102, 375.13, 0.01145, 13.03, and 31% correspondingly. PBOM8_13bits emerges as the design with the lowest resource consumption. PBOM8_13bits exhibits significant improvements in the area (88.98%), power consumption (96.195%), critical path delay (70%), PDP (98.96%), and PADP (99.87%). However, PBOM8_13bits shows the highest values for MRED (0.1537), MED (3166.5), NMED (0.0966), NoEB (3.49) and ER (87%). PBOM8_hybrid has enhanced area, power consumption, critical path delay, PDP, and PADP by 50%, 51.11%, 7.01%, 54.5%, and 77.26% respectively. However, MRED, MED, NMED, MSED, NMSED,NoEB and ER values for Hybrid are 0.024464, 221, 0.00675, 23020.5, 0.7068, 7.7, and 85% in their given order. PBOM8_73Y is the best design because it has an acceptable result for the least area, power, and delay. PBOM8_73Y has improved area, power, delay, PDP, and PADP by 63.47%, 71.37%, 14%, 75.38%, and 91% respectively. On the other hand, values of MRED, MED, NMED, MSED, NMSED,NoEB and ER are 0.05333, 579.69, 0.01769, 25230.3, 0.76997, 6.17 and 84% respectively. In PDEP and PADEP calculation, an error is reflected by MRED. PBOM8_30N shows the lowest PDEP and PADEP (0.02799 and 12.248) due to minimal MRED. Hence, PBOM8 shows significant enhancement in hardware efficiency compared to Exact_8bits.

VI. Comparison and Discussion

Table V also compares the hardware efficiency and error characteristics of the PBOM8 design with the literature. PBOM8 exhibits improvements in the area, power consumption, critical path delay, PDP, PADP and MRED are 41.68%, 73.16%, 35.57%, 72.65%, 75.42%, and 9.25% in their respective order on average. Figures 8 and 9 depict hardware and error analyses of PBOM8. The proposed design compared with the latest literature [3], [5], [6], [7], [8], [9], [10], [17], [18], [24], [25], [31], [32], [34], [35], [39], [44], [45], [46]. These designs are implemented using either of one technology, including 40nm, 32nm, 45nm, and 28nm.

Implemented designs PBOM8_9bits, PBOM8_10bits, PBOM8_11bits, PBOM8_12bits, and PBOM8_13bits exhibit the smallest area. Additionally, [3], [5], and [44] have improved mean area compared to the proposed design of 30.19%, 12.87%, and 7.66%. The implemented multipliers have enhanced delay, power, PDP, and PADP with respect to [5] are 46.5%, 84.8%, 89.48%, and 79.07% respectively on average. Recommended multipliers have amended delay, power, PDP, and PADP by 40%, 89.9%, 92%, and 86.4% respectively compared with [44] on average. In contrast, [5] and [44] have improved the mean of MRED by 89.78%, and 29.84%.

For delay comparison, references [6], [7], [25], [32], [39], and [41] have upgraded delay by 68.88%, 64.18%, 52.3%, 14.48%, 47.7% and 1.88% respectively on average. Compared with [6], presented designs have increased area, power, PDP, PADP, and MRED by 54.96%, 85.5%, 56.04%, 82.2%, and 76.3% respectively on average. Our designs have improved mean of area, power, PDP, and PADP by 56.55%, 95.98%, 85.9%, and 91.33% respectively compared with [7]. When juxtaposed with [39], the implemented multiplier has improved area, power, PDP, and PADP by 23.35%, 89%, 84.63%, and 82.9% respectively. The proposed designs have enhanced area, power, PDP, and PADP by 23.87%, 94.53%, 86.7%, 85.9%, and 98.34% against [25]. Investigated multipliers have augmented area, power, PDP, and PADP by 92.97%, 66.43%, 60%, and 94.4% compared with [32]. Relative to [31], our multipliers have increased area, power, delay, PDP, PADP, and MRED by 92.68%, 70.66%, 4.42%, 67.5%, 97%, and

TABLE V
HARDWARE PERFORMANCE AND ERROR COMPARISON FOR IMPLEMENTED DESIGNS AND MULTIPLIERS AVAILABLE IN LITERATURE

Design	Tech (nm)	Area (μm^2)	Power (μW)	Delay (ns)	PDP	PADP	PDEP	PADEP	MRED	MED	NMED	NoEB	ER (%)
Exact_8bits	40	481.9	52.57	0.57	29.96	14440	0	0	0	0	0	16	0
PBOM8_30N	40	437.47	48.156	0.57	27.4489	12008	0.02799	11.644	0.00102	3.3437	0.000102	13.03	31.64
PBOM8_50N	40	395	43.11	0.57	24.5727	9706.2	0.1020356	40.304	0.0041524	18.5312	0.0005655	10.86	46.68
PBOM8_33N	40	381	41.95	0.57	23.9115	9110.28	0.17455	66.505	0.0073	56.84	0.0017347	9.17	47.09
PBOM8_53N	40	331	35.855	0.57	20.4373	6764.76	0.2133	70.6038	0.010437	72.03	0.002198	8.99	58.68
PBOM8_73N	40	274	29.716	0.56	16.64	4559.623	0.2955	80.974	0.017759	132.78	0.004052	8.3	66.42
PBOM8_Hybrid	40	241	25.7	0.53	13.621	3282.661	0.333224	80.3070	0.024464	221	0.00674760	7.7	70.34
PBOM8_75N	40	241	26.34	0.51	13.43	3237.449	0.4892	117.899	0.0364175	375.78	0.0114679	6.71	72
PBOM8_105N	40	211	23.035	0.50	11.5175	2430.1925	0.4837	103.283	0.0422	451.78	0.013787	6.504	73.59
PBOM8_8bits	40	196	19.7	0.58	11.426	2239.496	0.55	107.94	0.0482	456.36	0.007	6.7	83.9
PBOM8_73Y	40	**176**	**15.05**	**0.49**	**7.3745**	**1297.912**	**0.393282**	**69.21764**	**0.05333**	**579.69**	**0.008914**	**6.17**	**84**
PBOM8_9bits	40	126.8	9.97	0.47	4.68	594	0.334	42.38	0.07135	803.49	0.0123	5.86	85.8
PBOM8_10bits	40	107.8	8.28	0.30	2.484	267.7752	0.2417	26.0598	0.09732	1302.49	0.02003	5.1	86.79
PBOM8_11bits	40	62.26	2.69	0.30	0.807	50	0.099	6.175	0.1229	1950.5	0.02999	4.43	87.29
PBOM8_12bits	40	61.9	3	0.16	0.48	29.712	0.06878	4.2577	0.1433	2654	0.04082	3.87	87.5
PBOM8_13bits	40	53.1	2	0.17	0.34	18	0.0522	2.766	0.1537	3166.5	0.04869	3.49	87.57
LOAM [24]	40	327	95	0.89	84.55	27647.85	1.53	500.426	0.0181		0.002	-	-
Trum_1 [38]	40	202.68	29.16	1.55	45.198	9160.73	-	-	-	127.3	-	-	74.6
MGER_4g [39]	40	350	255	0.39	99.45	34807.5	1.46055	511.1925	0.0107	-	0.0025	-	-
CDM8_a6 [3]	45	193.1	55.601	0.502	27.912	5390	2.108	407.034	0.0766	777.25	0.01195	5.81	73.6
k=4(H) [11]	45	451.7	96.2	0.61	58.682	26506.659	-	-	-	-	0.0135	-	-
TL8-4/3 [8]	45	313.08	12.10	0.88	10.65	3333.45	1.4299	447.68	0.1343	-	0.023	-	-
Ax8-3 [18]	45	301.65	143.54	0.77	110.64	33374.56	944.5	3.13	0.0283	-	0.00612	-	69.73
Jiang et al. [40]*	45	349.3	50.6	0.58	29.3	10251	7.42	2590.49	0.2527	1664.64	0.0256	-	-
Edavoor et al. [31]	45	2468	51.01	0.33	16.83	41536.44	0.81978	2023.22	0.0487	573.4	0.0027	-	-
C-SSM, m=6 [41]	28	153.9	147.8	0.242	35.7676	5504.634	0.78688	121.101	0.022	-	0.0306	-	-
mul2 [6]	28	318.5	73.4	0.10	7.3	2325.05	1.1023	351.08	0.151	-	0.017	-	98.86
Jiang et al. [42]*	28	301.6	143.5	0.77	110.5	33340	3.13	943.52	0.0283	397.95	0.00612	-	69.73
Ansari et al. [43]*	28	217.3	40	0.53	21.2	4607	1.71	371.76	0.0807	578.72	0.0089	-	-
Rashidi et al. [20]	40	483.84	18.63	2.21	41.17	19920.8	6.596	3191.31	0.1602	2514	0.0387	3.238	85.56

Note: (*) Reported by [3].

53.27%. While independent works of literatures [5], [6], [7], [41], and [44] have been designed on 28nm.

Hence, we can conclude that literatures [31] and [32] are the highest area designs and it is out of the plotted range (Figure 8(b)). Reference [34] is having maximum delay (Figure 8(c)). The highest power designs are [7], [25], and [35] (Figure 8(a)). Compared to all conventional multipliers, PBOM8 demonstrates superiority in terms of power Figure 8(a), PDP Figure 9(a), and PADP Figure 9(b). Along with it, PBOM8 outperforms all conventional multipliers in either area, delay, MRED or in some cases excels in all these aspects. Therefore, it can be concluded that overall PBOM8 approximate multipliers significantly improved hardware efficiency as well as accuracy.

VII. APPLICATION: IMAGE PROCESSING AND NEURAL NETWORK

We have tested the merit of the investigated multipliers using image processing and deep neural networks. All proposed multipliers are executed for image smoothing, edge detection, image sharpening, and deep neural networks. These applications aid in delineating the proposed design's scope of applicability.

A. Image Processing

We employed three filters during our experiment: Gaussian, Sobel, and Laplacian. This study aims to explore, how the performance of smoothing, edge detection, and sharpening is influenced by progressively approximating more partial products. The PSNR and SSIM are used to measure the performance of the approximate multiplier. Typically, the PSNR metric is used for image quality value estimation. While PSNR only considers Mean Square Error (MSE), and it is sensitive to small changes in pixel values. It may not accurately represent the perceived quality of the image. However, SSIM explains the structural similarity of the image by analogizing local patterns of pixel intensities. Moreover, SSIM stands out as a metric that holds greater perceptual relevance when evaluating the quality of images. Hence, by taking both PSNR and SSIM into account in image processing, a more comprehensive assessment of image quality can be achieved. Table VI presents the image smoothing, edge detection, and image-sharpening PSNR and SSIM results for implemented designs and compared with the literatures [6], [41], [43], and [47].

1) Image Smoothing Using Gaussian Filter: Gaussian smoothing, a prevalent technique for image blurring finds

TABLE VI
PERFORMANCE COMPARISON OF PROPOSED MULTIPLIERS IN IMAGE PROCESSING AND NEURAL NETWORK

Design	Smoothing		Edge Detection				Sharpening				DNN
Multiplier	Gaussian filter		Sobel filter (3x3)		Sobel filter (5x5)		Laplacian filter (3x3)		High Pass filter (3x3)		MNIST
	PSNR(dB)	SSIM	PSNR(dB)	SSIM	PSNR(dB)	SSIM	PSNR(dB)	SSIM	PSNR(dB)	SSIM	Accuracy
PBOM8_30N	60.6	0.999	∞	1	58.35	0.999	∞	1	27	0.95	96.96
PBOM8_50N	53.18	0.999	∞	1	47.08	0.999	∞	1	17.38	0.90	96.95
PBOM8_33N	48.50	0.999	∞	1	58.36	0.999	∞	1	27	0.95	96.94
PBOM8_53N	46.54	0.999	∞	1	47.08	0.999	∞	1	17.38	0.90	96.94
PBOM8_73N	41.452	0.998	∞	1	36.65	0.998	∞	1	9.708	0.342	96.93
PBOM8_Hybrid	39.02	0.99	∞	1	36.65	0.998	∞	1	9.708	0.342	96.93
PBOM8_75N	33.124	0.997	∞	1	32.29	0.986	∞	1	9.708	0.342	96.91
PBOM8_105N	33.11	0.987	∞	1	31.78	0.961	∞	1	9.708	0.342	96.81
PBOM8_8bits	32.6	0.977	∞	1	32.29	0.986	∞	1	9.708	0.342	96.73
PBOM8_73Y	31.7	0.96	∞	1	36.65	0.998	∞	1	9.708	0.342	96.73
PBOM8_9bits	27.4	0.953	∞	1	31.78	0.961	∞	1	9.708	0.342	96.09
PBOM8_10bits	23.76	0.881	∞	1	31.78	0.961	∞	1	9.708	0.342	95.31
PBOM8_11bits	19.17	0.823	∞	1	31.78	0.961	∞	1	9.708	0.342	95.31
PBOM8_12bits	17.5	0.799	∞	1	31.78	0.961	∞	1	9.708	0.342	95.13
PBOM8_13bits	20.2	0.799	∞	1	31.78	0.961	∞	1	9.708	0.342	95.13
Proposed-mul1 [6]	30.04	0.951	39.98	0.81	-	-	28.09	0.984	-	-	91.1
Proposed-mul2 [6]	26.24	0.956	46.36	0.973	-	-	29.32	0.986	-	-	91.3
Mul-1 [47]*	30.62	0.941	42.92	0.948	-	-	31.23	0.987	-	-	91.0
Mul-2 [47]*	33.95	0.984	42.67	0.947	-	-	33.95	0.988	-	-	90.9
M8-1 [43]*	31.36	0.95	44.05	0.962	-	-	32.1	0.991	-	-	91.1
M8-2 [43]*	39.81	0.984	41.24	0.928	-	-	30.05	0.986	-	-	91.2
SSM,m=4 [41]	25	0.979	-	-	10.7	0.16	-	-	-	-	29.67
C-SSM,m=4 [41]	31.9	0.991	-	-	13.5	0.38	-	-	-	-	90.58
SSM,m=5 [41]	29.3	0.994	-	-	38.7	0.95	-	-	-	-	95.90
C-SSM,m=5 [41]	42.5	0.996	-	-	36.9	0.93	-	-	-	-	96.85
SSM,m=6 [41]	38.4	0.998	-	-	45.1	0.99	-	-	-	-	97.15
C-SSM,m=6 [41]	50.0	0.998	-	-	43.7	0.98	-	-	-	-	97.21

Note: (*) Reported by [6].

widespread use in error-resilient applications. Numerous studies have assessed the efficacy of their designs under this configuration. The standard deviation of the Gaussian distribution determines the degree of smoothing. Gaussian filter h_1 and modified Gaussian filter h_2 of size 3×3 are given as follows:

$$h1 = \begin{bmatrix} 0.095 & 0.118 & 0.095 \\ 0.118 & 0.148 & 0.118 \\ 0.095 & 0.118 & 0.095 \end{bmatrix}$$
$$h2 = \begin{bmatrix} 95 & 118 & 95 \\ 118 & 148 & 118 \\ 95 & 118 & 95 \end{bmatrix} \quad (8)$$

Table VI portrays the PSNR and SSIM values for Gaussian smoothing employing implemented multipliers and existing literature. Notably, increasing levels of approximation correlate with diminishing PSNR and SSIM values. Figure 10 depicts output images of Gaussian smoothing for Barbara's original image using proposed 8-bit approximate multipliers. Except for PBOM8_9bits, PBOM8_10bits, PBOM8_11bits, PBOM8_12bits, and PBOM8_13bits the other multipliers have acceptable PSNR and SSIM over 30 dB and 94% respectively. PBOM8_30N has the highest PSNR (60.6dB) and SSIM (99.999%). The quality degradation of images visible in Figure 10(b) and 10(c), 10(d), 10(e), 10(f), 10(g), 10(h), 10(i), 10(j), 10(k), is almost negligible, because it is having PSNR and SSIM >30 dB, 96%. Figures 10(l), and 10(m) show a slight deterioration in image quality due to their PSNR being less than 30 dB. In Figure 10(n), 10(o) has significant quality degradation because of the least PSNR and SSIM values. PBOM8_50N which approximates five columns of multiplier-1 and PBOM8_33N which approximates three columns of both multipliers. PBOM8_33 has more accuracy degradation (compared to PBOM8_50N), that depicts multiplier-1 has more impact on the quality degradation of images compared to multiplier-2. In summary, Table VI and Figure 10 demonstrate that an increase in approximated columns will degrade the performance of Gaussian smoothing. To increase hardware efficiency with acceptable performance, we recommend to use PBOM8_75N, PBOM8_10N, PBOM8_8bits, and PBOM8_73Y instead of PBOM8_(30, 50, 33, 53, 73)N. Hence, achieving satisfactory Gaussian smoothing performance with reduced hardware is viable.

2) Edge Detection Using Sobel Operator: Edge detection holds a variety of applications within the realm of computer vision serving as a fundamental technique for tasks, such as object recognition, image segmentation, feature extraction, and boundary detection. The Sobel operators with dimensions 3×3 and 5×5 [41] are given by:

$$S_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

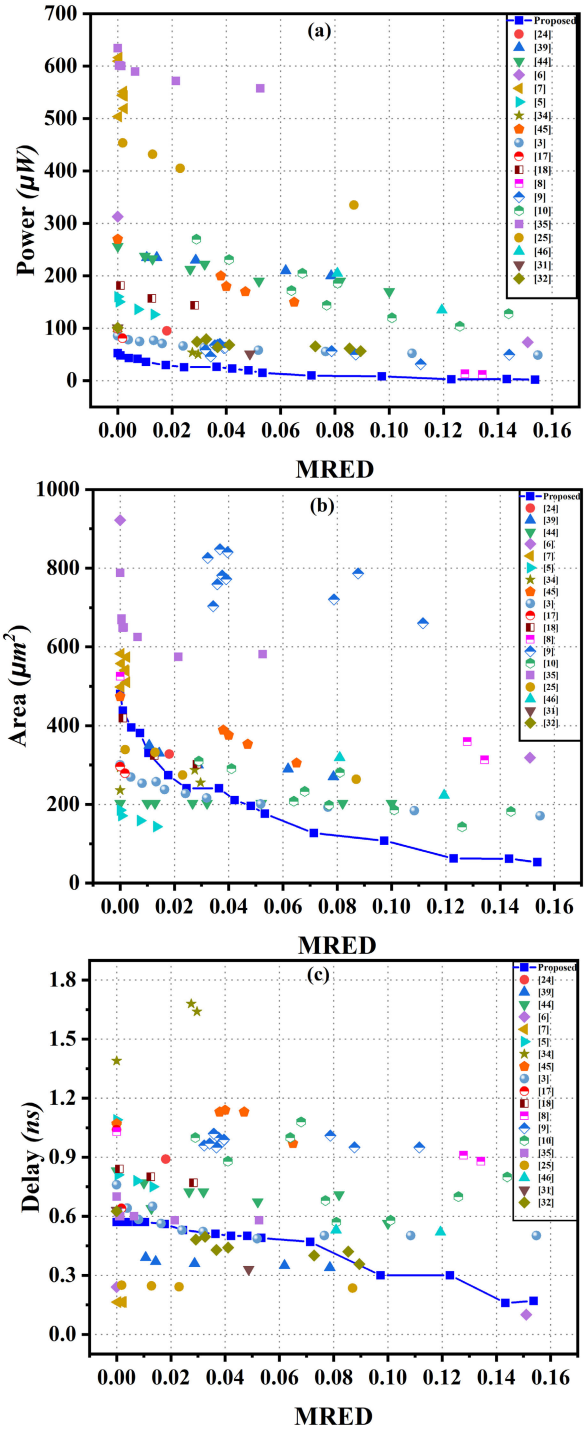

Fig. 8. Tradeoff analysis of (a) power, (b) area, and (c) delay versus MRED for investigated 8× 8 designs compared with literature.

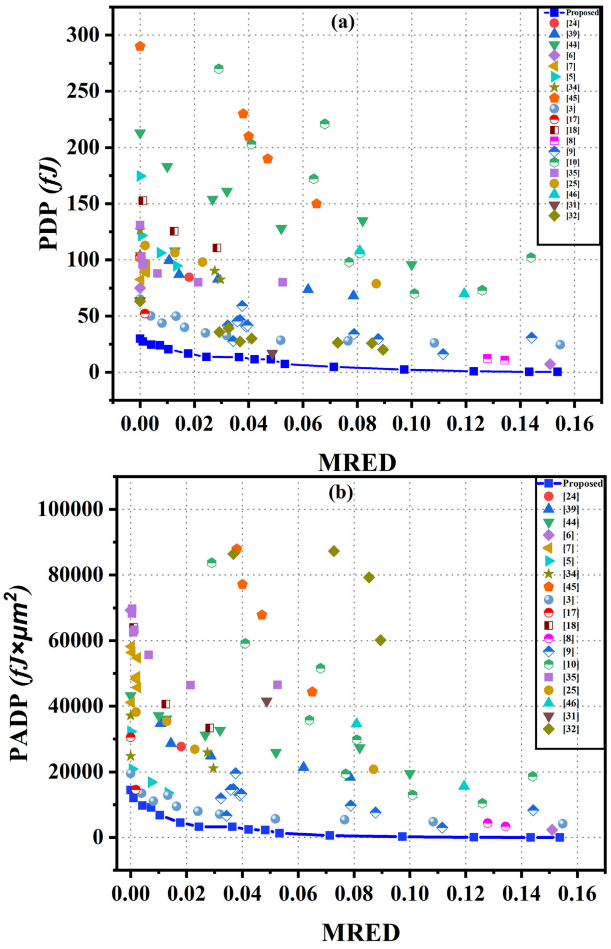

Fig. 9. Tradeoff analysis of (a) PDP, and (b) PADP versus MRED for proposed 8 × 8 multipliers compared with literature.

$$S_x = \begin{bmatrix} 1 & 2 & 0 & -2 & -1 \\ 4 & 8 & 0 & -8 & -4 \\ 6 & 12 & 0 & -12 & -6 \\ 4 & 8 & 0 & -8 & -4 \\ 1 & 2 & 0 & -2 & -1 \end{bmatrix} \tag{9}$$

The y component of kernel S_y is the transpose of S_x. Figure 11 and Table VI illustrate the PSNR and SSIM edge detection results for the "cameraman" image using the Sobel operator in equation (17). Corresponding to 3 × 3 Sobel operator has values (−1, 1, −2, 2, 0). All elements can be expressed by 2^n, which can be verified by Figure 11(b). Designed multipliers provide exceptionally high PSNR and SSIM leads to the exact results. Sobel operator with size 5 × 5, all elements we can express in the form of 2^n except (−6, 6, −12, 12). Due to this, there is a degradation in PSNR and SSIM values, depicted in Figure 11((d), (e), (f), (g), (h), (i), (j)). Image edge width of 3 × 3 (Figure 11(b)) is lesser compared to 5 × 5. Using the 5 × 5 Sobel operator, edge detection achieved having acceptable PSNR >31 dB and SSIM >96% for all proposed multipliers. Increased approximation degrades PSNR and SSIM, but visually the quality degradation is not noticeable (Figure 11). Hence, PBOM8_13bits (Figure 11(j)) is suggested for edge detection over exact or complex multipliers.

3) Image Sharpening Using Laplacian Filter and High Pass Filter (HPF): Sharpening enhances edge clarity in images by accentuating intensity and contrast changes. A common form of Laplacian filter denoted as L_1, and an HPF denoted as L_2, both with sizes of 3 × 3 are provided as follows:

$$L_1 = \begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

$$L_2 = \begin{bmatrix} -1 & -1 & -1 \\ -1 & 9 & -1 \\ -1 & -1 & -1 \end{bmatrix} \tag{10}$$

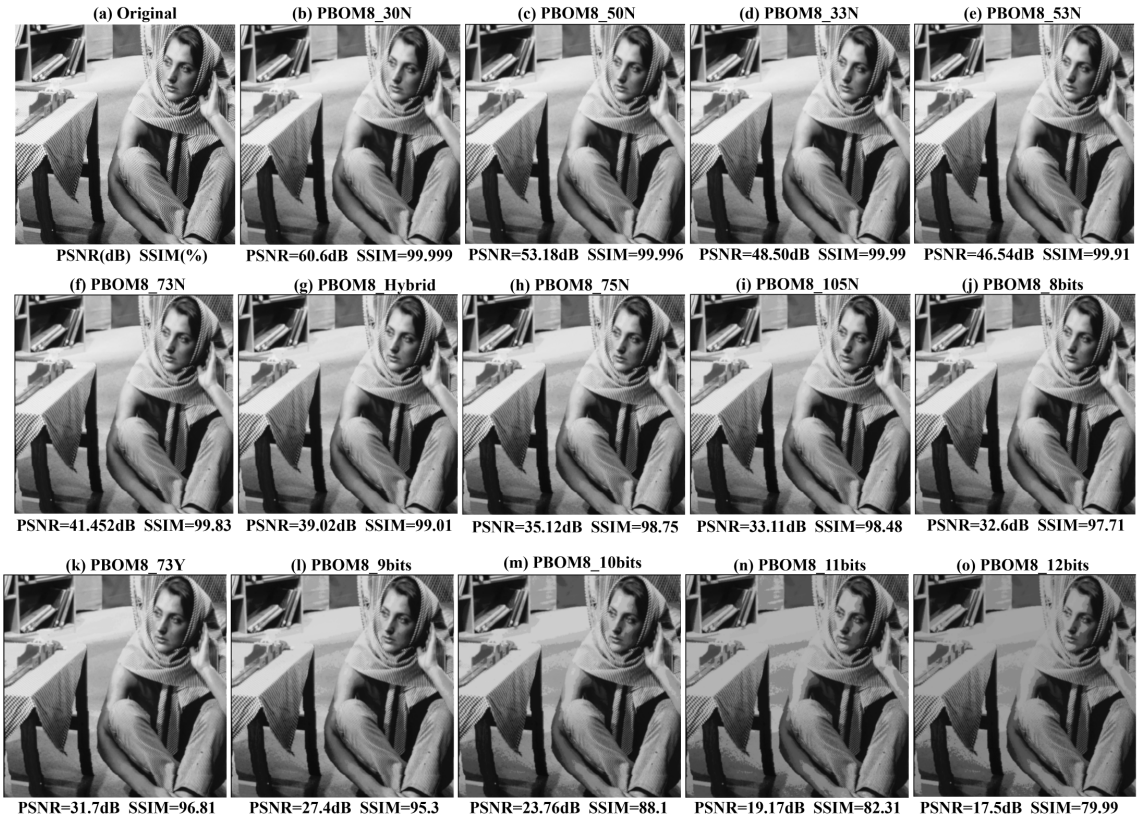

Fig. 10. Image smoothing results achieved for Barbara's original image using the designated multipliers.

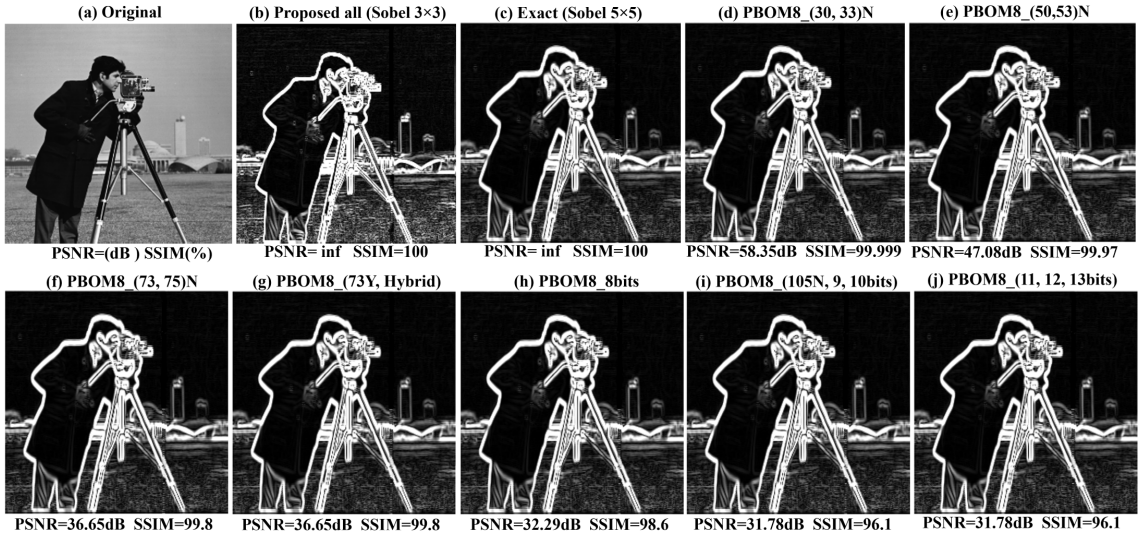

Fig. 11. Sobel edge detection results obtained for the cameraman image using our specialized approximate multipliers.

Utilizing the 3× 3 Laplacian operator with values (−1, 8), all elements can be expressed as 2^n. As an output, it gives exact results for all proposed multipliers, which is reported in Table VI with PSNR tending to infinite and SSIM 100%. However, for the HPF operator, except for element value 9, all elements can be configured as 2^n. Due to this, there is a reduction in PSNR and SSIM. Hence, sharpening using the Laplacian operator is preferable to HPF for implemented designs.

B. Deep Neural Network Application

Another application of speculated multipliers is the hardware implementation of a deep neural network (DNN). The fully connected neural network consists of an input layer with 784 neurons, three hidden layers with 30, 20, and 10 neurons utilizing the sigmoid activation function, and an output layer comprising 10 neurons representing each numeric digit. The network was trained using the MNIST dataset [48],

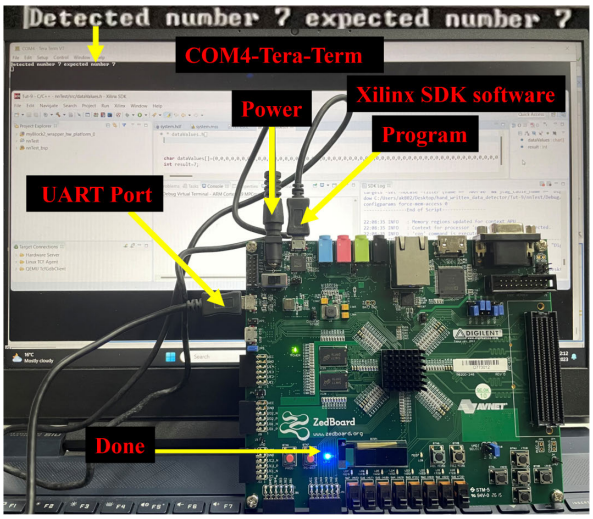

Fig. 12. Illustration of hand-written digit detection using PBOM8_13bits approximate multiplier on Zed board.

which contains 60,000 (images) handwritten digits for training and 10,000 handwritten digits for testing the model. The architectures of the deep learning model are formulated using Python programming language. While training, its corresponding weights and bias values are stored in the weight and bias memory. Weights and biases can be negative or positive. These stored weights and biases are then utilized as a Read-Only Memory (ROM) for hardware implementation. Additionally, bias is added when the last multiplication is performed. The total sum is then passed through the sigmoid activation function. All the proposed multipliers are performed on a given neural network. The accuracy is used as a metric to evaluate correct recognition. The last column of Table VI displays the accuracy of this implementation. PBOM8_30N has the same accuracy as in exact multiplier which is 96.96%. PBOM8_13 bits have the least accuracy which drops by 1.8% compared to exact_8bits. The performance using the investigated approximate multipliers indicates only a negligible drop in accuracy. Investigated design PBOM8_13bits is deployed on ZedBoard Zynq-7000. Figure 12 demonstrates that the expected and detected numbers match with a value of 7 on the ZedBoard Zynq-7000 board.

VIII. Conclusion

This study investigated fifteen types of approximate multipliers: eight employing recursive methods for less error-resilient applications, six utilizing bit-wise approximation for high error-tolerant applications, and one applying a hybrid approach for balanced error. Compared to the accurate multiplier, significant improvements in area, power, delay, PDP, and PADP by 41.68%, 73.16%, 35.57%, 72.65%, and 75.42% on average. In the existing multipliers, enhancements in area, power, delay, PDP, and PADP are 54.41%, 57.57%, 25.73%, 60.14%, and 74.33% respectively. On contrasting the performance of existing and designed multipliers using trade-off plots, least power, PDP, and PADP were observed. Additionally, for bit-wise approximation design, it attains the smallest area. Most of the multipliers demonstrated acceptable performance in image smoothing, surpassing PSNR and SSIM values of 30dB and 94% correspondingly. Edge detection and image sharpening using 3×3 Sobel operator and Laplacian filter resulted in outstanding PSNR and SSIM values tending to infinity and 100% respectively. When employing the 5×5 Sobel operator for edge detection, all recommended multipliers yielded PSNR and SSIM values of more than 31dB and 96%. Additionally, in image sharpening using HPF, a PSNR of 27dB and SSIM of 95% were attained. Furthermore, implementing a DNN using the MNIST dataset with the proposed multipliers achieved an accuracy of more than 95%. Our future plans involve integrating approximate multiply and accumulate (MAC) units into Reduced Instruction Set Computing (RISC-V) processors for various AI/ML applications. We anticipate that this approach will significantly enhance the performance of RISC-V in tasks related to image processing and advancing AI/ML technology.

References

[1] M. H. Haider and S.-B. Ko, "Booth encoding-based energy efficient multipliers for deep learning systems," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 70, no. 6, pp. 2241–2245, Jun. 2023.

[2] W. Liu, F. Lombardi, and M. Shulte, "A retrospective and prospective view of approximate computing [point of view," *Proc. IEEE*, vol. 108, no. 3, pp. 394–399, Mar. 2020.

[3] N. Amirafshar, A. S. Baroughi, H. S. Shahhoseini, and N. TaheriNejad, "Carry disregard approximate multipliers," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 70, no. 12, pp. 4840–4853, Dec. 2023.

[4] B. K. Mohanty, "Efficient approximate multiplier design based on hybrid higher radix booth encoding," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 13, no. 1, pp. 165–174, Mar. 2023.

[5] H. Waris, C. Wang, C. Xu, and W. Liu, "AxRMs: Approximate recursive multipliers using high-performance building blocks," *IEEE Trans. Emerg. Topics Comput.*, vol. 10, no. 2, pp. 1229–1235, Apr. 2022.

[6] L. Sayadi, S. Timarchi, and A. Sheikh-Akbari, "Two efficient approximate unsigned multipliers by developing new configuration for approximate 4:2 compressors," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 70, no. 4, pp. 1649–1659, Apr. 2023.

[7] T. Kong and S. Li, "Design and analysis of approximate 4–2 compressors for high-accuracy multipliers," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 29, no. 10, pp. 1771–1781, Oct. 2021.

[8] R. Pilipovic, P. Bulic, and U. Lotric, "A two-stage operand trimming approximate logarithmic multiplier," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 68, no. 6, pp. 2535–2545, Jun. 2021.

[9] W. Liu, J. Xu, D. Wang, C. Wang, P. Montuschi, and F. Lombardi, "Design and evaluation of approximate logarithmic multipliers for low power error-tolerant applications," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 65, no. 9, pp. 2856–2868, Sep. 2018.

[10] S. Vahdat, M. Kamal, A. Afzali-Kusha, and M. Pedram, "TOSAM: An energy-efficient truncation- and rounding-based scalable approximate multiplier," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 27, no. 5, pp. 1161–1173, May 2019.

[11] K. Chen, W. Liu, J. Han, and F. Lombardi, "Profile-based output error compensation for approximate arithmetic circuits," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 67, no. 12, pp. 4707–4718, Dec. 2020.

[12] A. Akbar Bahoo, O. Akbari, and M. Shafique, "An energy-efficient generic accuracy configurable multiplier based on block-level voltage overscaling," *IEEE Trans. Emerg. Topics Comput.*, vol. 11, no. 4, pp. 851–867, Dec. 2023.

[13] F. Guella, E. Valpreda, M. Caon, G. Masera, and M. Martina, "MARLIN: A co-design methodology for approximate ReconfigurabLe inference of neural networks at the edge," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 71, no. 5, pp. 2105–2118, May 2024.

[14] A. S. Baroughi, S. Huemer, H. S. Shahhoseini, and N. TaheriNejad, "AxE: An approximate-exact multi-processor system-on-chip platform," in *Proc. 25th Euromicro Conf. Digit. Syst. Design (DSD)*, Aug. 2022, pp. 60–66.

[15] S. Xu and B. C. Schafer, "Approximate reconfigurable hardware accelerator: Adapting the micro-architecture to dynamic workloads," in *Proc. IEEE Int. Conf. Comput. Design (ICCD)*, Nov. 2017, pp. 113–120.

[16] H. Jiang, F. J. H. Santiago, H. Mo, L. Liu, and J. Han, "Approximate arithmetic circuits: A survey, characterization, and recent applications," *Proc. IEEE*, vol. 108, no. 12, pp. 2108–2135, Dec. 2020.

[17] N. Amirafshar, A. S. Baroughi, H. S. Shahhoseini, and N. TaheriNejad, "An approximate carry disregard multiplier with improved mean relative error distance and probability of correctness," in *Proc. 25th Euromicro Conf. Digit. Syst. Design (DSD)*, Los Alamitos, CA, USA, Aug. 2022, pp. 46–52, doi: 10.1109/DSD57027.2022.00016.

[18] H. Waris, C. Wang, W. Liu, J. Han, and F. Lombardi, "Hybrid partial product-based high-performance approximate recursive multipliers," *IEEE Trans. Emerg. Topics Comput.*, vol. 10, no. 1, pp. 507–513, Jan. 2022.

[19] K. Bhardwaj, P. S. Mane, and J. Henkel, "Power- and area-efficient approximate Wallace tree multiplier for error-resilient systems," in *Proc. 15th Int. Symp. Quality Electron. Design*, Mar. 2014, pp. 263–269.

[20] B. Rashidi, "Efficient and low-cost approximate multipliers for image processing applications," *Integration*, vol. 94, Jan. 2024, Art. no. 102084. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0167926023001268

[21] S. Rehman, W. El-Harouni, M. Shafique, A. Kumar, J. Henkel, and J. Henkel, "Architectural-space exploration of approximate multipliers," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD)*, Nov. 2016, pp. 1–8.

[22] P. Kulkarni, P. Gupta, and M. Ercegovac, "Trading accuracy for power with an underdesigned multiplier architecture," in *Proc. 24th Int. Conf. VLSI Design*, 2011, pp. 346–351.

[23] E. Zacharelos, I. Nunziata, G. Saggese, A. G. M. Strollo, and E. Napoli, "Approximate recursive multipliers using low power building blocks," *IEEE Trans. Emerg. Topics Comput.*, vol. 10, no. 3, pp. 1315–1330, Jul. 2022.

[24] Y. Guo, H. Sun, and S. Kimura, "Design of power and area efficient lower-part-OR approximate multiplier," in *Proc. IEEE Region Conference*, 2018, pp. 2110–2115.

[25] S. K. N. Mahammad, "Low power, high speed approximate multiplier for error resilient applications," *Integration*, vol. 84, pp. 37–46, May 2022. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0167926022000013

[26] M. Ha and S. Lee, "Multipliers with approximate 4–2 compressors and error recovery modules," *IEEE Embedded Syst. Lett.*, vol. 10, no. 1, pp. 6–9, Mar. 2018.

[27] U. A. Kumar, S. V. Bharadwaj, A. B. Pattaje, S. Nambi, and S. E. Ahmed, "CAAM: Compressor-based adaptive approximate multiplier for neural network applications," *IEEE Embedded Syst. Lett.*, vol. 15, no. 3, pp. 117–120, Aug. 2023.

[28] Y. Chang, Y. Cheng, Y. Lin, S. Liao, C. Lai, and T. Wu, "Imprecise 4–2 compressor design used in image processing applications," *IET Circuits, Devices Syst.*, vol. 13, no. 6, pp. 848–856, Sep. 2019. [Online]. Available: https://ietresearch.onlinelibrary.wiley.com/doi/abs/10.1049/iet-cds.2018.5403

[29] U. A. Kumar, R. K. Chintakunta, S. Kumar, K. Jamal, and S. E. Ahmed, "Approximate multiplier architectures for error resilient applications," in *Proc. IEEE Int. Symp. Smart Electron. Syst. (iSES)*, Dec. 2021, pp. 89–92.

[30] D. Esposito, A. G. M. Strollo, E. Napoli, D. De Caro, and N. Petra, "Approximate multipliers based on new approximate compressors," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 65, no. 12, pp. 4169–4182, Dec. 2018.

[31] P. J. Edavoor, S. Raveendran, and A. D. Rahulkar, "Approximate multiplier design using novel dual-stage 4:2 compressors," *IEEE Access*, vol. 8, pp. 48337–48351, 2020.

[32] K. Manikantta Reddy, M. H. Vasantha, Y. B. Nithin Kumar, and D. Dwivedi, "Design and analysis of multiplier using approximate 4–2 compressor," *AEU-Int. J. Electron. Commun.*, vol. 107, pp. 89–97, Jul. 2019. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1434841118330085

[33] P. Yin, C. Wang, H. Waris, W. Liu, Y. Han, and F. Lombardi, "Design and analysis of energy-efficient dynamic range approximate logarithmic multipliers for machine learning," *IEEE Trans. Sustain. Comput.*, vol. 6, no. 4, pp. 612–625, Oct. 2021.

[34] M. S. Ansari, B. F. Cockburn, and J. Han, "An improved logarithmic multiplier for energy-efficient neural computing," *IEEE Trans. Comput.*, vol. 70, no. 4, pp. 614–625, Apr. 2021.

[35] W. Liu, L. Qian, C. Wang, H. Jiang, J. Han, and F. Lombardi, "Design of approximate radix-4 booth multipliers for error-tolerant computing," *IEEE Trans. Comput.*, vol. 66, no. 8, pp. 1435–1441, Aug. 2017.

[36] F. Sabetzadeh et al., "An ultra-efficient approximate multiplier with error compensation for error-resilient applications," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 70, no. 2, pp. 776–780, Feb. 2023.

[37] Z. Ebrahimi, M. Zaid, M. Wijtvliet, and A. Kumar, "RAPID: Approximate pipelined soft multipliers and dividers for high throughput and energy efficiency," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 42, no. 3, pp. 712–725, Mar. 2023.

[38] H. Mo et al., "Learning the error features of approximate multipliers for neural network applications," *IEEE Trans. Comput.*, vol. 73, no. 3, pp. 842–856, Mar. 2024.

[39] Y. Guo, H. Sun, L. Guo, and S. Kimura, "Low-cost approximate multiplier design using probability-driven inexact compressors," in *Proc. IEEE Asia–Pacific Conf. Circuits Syst. (APCCAS)*, Oct. 2018, pp. 291–294.

[40] H. Jiang, S. Angizi, D. Fan, J. Han, and L. Liu, "Non-volatile approximate arithmetic circuits using scalable hybrid spin-CMOS majority gates," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 68, no. 3, pp. 1217–1230, Mar. 2021.

[41] A. G. M. Strollo, E. Napoli, D. De Caro, N. Petra, G. Saggese, and G. Di Meo, "Approximate multipliers using static segmentation: Error analysis and improvements," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 69, no. 6, pp. 2449–2462, Jun. 2022.

[42] H. Jiang, C. Liu, L. Liu, F. Lombardi, and J. Han, "A review, classification, and comparative evaluation of approximate arithmetic circuits," *ACM J. Emerg. Technol. Comput. Syst.*, vol. 13, no. 4, pp. 1–34, Aug. 2017, doi: 10.1145/3094124.

[43] M. S. Ansari, H. Jiang, B. F. Cockburn, and J. Han, "Low-power approximate multipliers using encoded partial products and approximate compressors," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 8, no. 3, pp. 404–416, Sep. 2018.

[44] F. Frustaci, S. Perri, P. Corsonello, and M. Alioto, "Approximate multipliers with dynamic truncation for energy reduction via graceful quality degradation," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 67, no. 12, pp. 3427–3431, Dec. 2020.

[45] M. S. Kim, A. A. D. Barrio, L. T. Oliveira, R. Hermida, and N. Bagherzadeh, "Efficient Mitchell's approximate log multipliers for convolutional neural networks," *IEEE Trans. Comput.*, vol. 68, no. 5, pp. 660–675, May 2019.

[46] O. Akbari, M. Kamal, A. Afzali-Kusha, and M. Pedram, "Dual-quality 4:2 compressors for utilizing in dynamic accuracy configurable multipliers," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 25, no. 4, pp. 1352–1361, Apr. 2017.

[47] M. Ahmadinejad, M. H. Moaiyeri, and F. Sabetzadeh, "Energy and area efficient imprecise compressors for approximate multiplication at nanoscale," *AEU-Int. J. Electron. Commun.*, vol. 110, Oct. 2019, Art. no. 152859. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1434841119304923

[48] Y. LeCun, C. Cortes, and C. Burges. *MNIST Handwritten Digit Database*. Accessed: Sep. 27, 2023. [Online]. Available: http://yann.lecun.com/exdb/mnist

Anupam Kumari received the B.Tech. degree in electronics and communication engineering from Bhagalpur College of Engineering, Bhagalpur, India, in 2018, and the M.Tech. degree in embedded system design from NIT Kurukshetra, Kurukshetra, India, in 2021. She is currently pursuing the Ph.D. degree with the Indian Institute of Technology Guwahati, Guwahati, India. Her research interests include VLSI circuits for approximate computing, AI/ML, RISC-V processor, and the IoT hardware security.

Roy Paily Palathinkal (Senior Member, IEEE) received the B.Tech. degree in electronics and communication engineering from the College of Engineering, Thiruvananthapuram, India, in 1990, the M.Tech. degree from IIT Kanpur, Kanpur, India, in 1996, and the Ph.D. degree from IIT Madras, Chennai, India, in 2004. He is currently a Professor of electronics and electrical engineering with the Centre for Nanotechnology, and Jyoti and Bhupat Mehta School of Health Sciences and Technology, Indian Institute of Technology Guwahati, Guwahati, India. His research interests include VLSI devices/circuits and MEMS.